

Task 2: Intelligent Multi-Class Natural Language Text Sentiment Classifier

Progree Remote Internship: Artificial Intelligence
Intern: **Darakhshan Mujtaba**

1. Objective

The goal of this task was to build a program that reads short, unstructured text and classifies it into one of three sentiment classes: **negative**, **neutral** or **positive**. The text is preprocessed with stop-word removal and lemmatization, converted to numbers with TF-IDF, classified with Scikit-Learn models, and evaluated with F1 scores and a confusion matrix. All code was written and run in Google Colab.

2. Dataset

I used the public **TweetEval sentiment** dataset (Barbieri et al., 2020), which contains English tweets labelled by humans as negative (0), neutral (1) or positive (2). Tweets are a challenging source of text because they are short and informal, with slang, hashtags, mentions and sarcasm.

Split	Tweets	Negative	Neutral	Positive
Train	45,615	7,093	20,673	17,849
Test	12,284	3,972	5,937	2,375

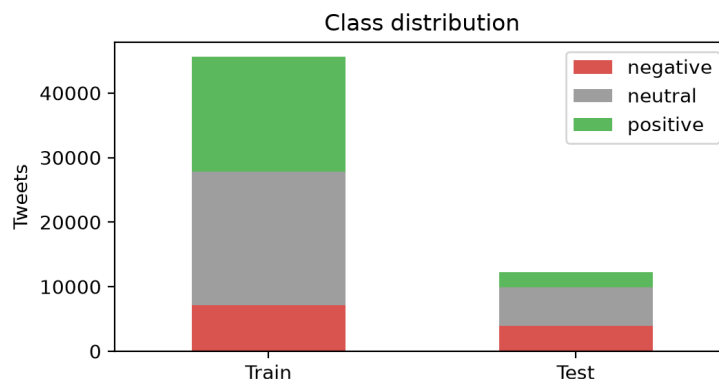


Figure 1. Number of tweets per class in the training and test sets.

The classes are imbalanced, so **macro-averaged F1**, which treats every class equally, is used as the main score.

3. Methodology

3.1 Text preprocessing

- Lower-casing, fixing escaped apostrophes, and replacing links and @mentions with the words *url* and *user*.
- Splitting each tweet into words.
- **Stop-word removal** with the NLTK English list. The negation words *no*, *not*, *nor*, *never* are kept, because removing them would change the meaning ("not good" would become "good").

- **Lemmatization** with the NLTK WordNet lemmatizer, so *loved* becomes *love* and *movies* becomes *movie*.

Original tweet	After preprocessing
Chris Mortensen reported that a deal is close to being done between NFL and the Refs. Could return on Sunday. Thank. God.	chris mortensen report deal close do nfl ref could return sunday thank god
Interesting to see Cliff Avril start. Told me Wednesday he wasn't sure if he was ready to go. Obviously he is.	interest see cliff avril start tell wednesday sure ready go obviously
The Vamps are on TFI Friday and look about 14 years old...	vamp tfi friday look year old
Adam Levine strikes a pose for photographers outside of the Late Show with David Letterman on Thursday (October 25) in New York City	adam levine strike pose photographer outside late show david letterman thursday october new york city

Table 2. Examples of tweets before and after preprocessing.

3.2 TF-IDF features

The cleaned text was converted into numbers with Scikit-Learn's `TfidfVectorizer`, using single words and two-word phrases (bigrams such as *not good*). Terms seen in fewer than 2 tweets or in more than 90% of tweets were ignored. Each tweet became a vector of **63,492 features**.

3.3 Models

- **Multinomial Naive Bayes** ($\alpha = 0.1$): a fast, probability-based baseline.
- **Logistic Regression** ($C = 2$, balanced class weights).
- **Linear SVM** ($C = 0.05$, balanced class weights).

Balanced class weights make the models pay more attention to the smaller negative class. Each model was trained on the training set and evaluated once on the separate test set.

4. Results

4.1 Model comparison

Model	Accuracy	Macro-F1	F1 negative	F1 neutral	F1 positive
Naive Bayes	0.563	0.534	0.464	0.630	0.507
Logistic Regression	0.593	0.587	0.611	0.594	0.557
Linear SVM	0.607	0.592	0.581	0.644	0.550

Table 3. Test-set scores of the three models.

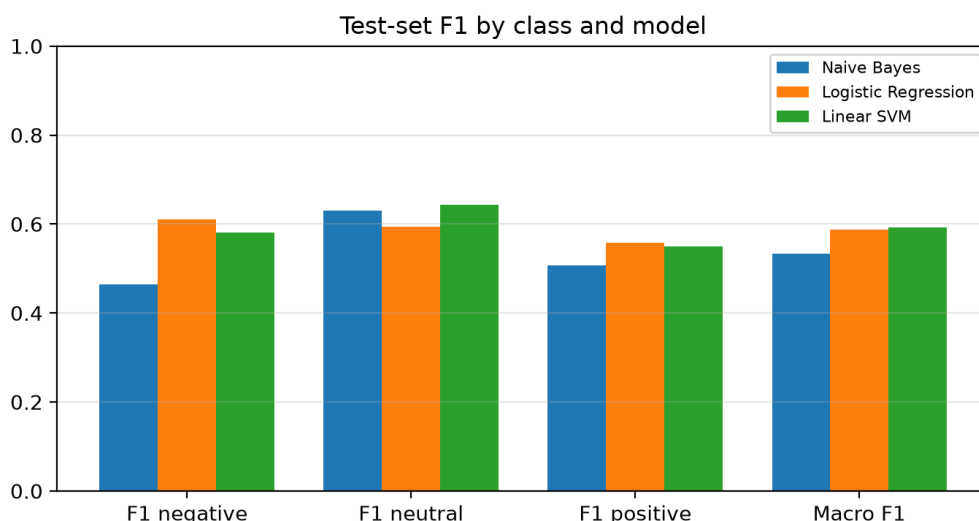


Figure 2. F1 score per class and macro-F1 for each model.

The **Linear SVM** performed best, with a macro-F1 of **0.592** and accuracy of 0.607. Logistic Regression was very close (0.587), while Naive Bayes was clearly weaker (0.534), mainly on the negative class.

4.2 Classification report and confusion matrix (Linear SVM)

Class	Precision	Recall	F1-score	Support
Negative	0.606	0.559	0.581	3,972
Neutral	0.620	0.670	0.644	5,937
Positive	0.570	0.531	0.550	2,375
Macro avg	0.599	0.586	0.592	12,284
Weighted avg	0.606	0.607	0.605	12,284

Table 4. Classification report on the test set (accuracy 0.607).

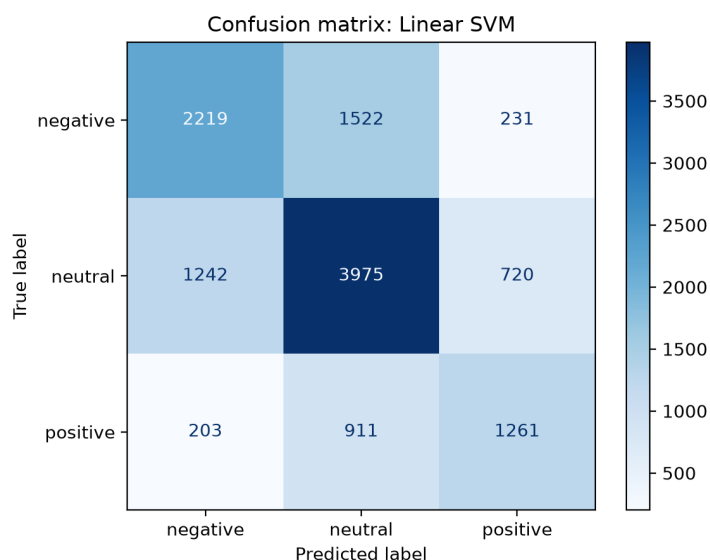


Figure 3. Confusion matrix of the Linear SVM on the test set.

Neutral is the easiest class (F1 0.644) and positive the hardest (0.550). The confusion matrix shows that most errors involve the neutral class: 1,522 negative and 911 positive tweets were predicted as neutral, and 1,242 neutral tweets as negative. Opposite opinions are rarely confused: only 231 negative tweets were predicted positive and 203 positive tweets negative. The hardest part of the problem is deciding

whether a mildly worded tweet expresses an opinion at all.

4.3 What the model learned

Class	Words with the highest weights
Negative	f***, worst, s***, sad, not, hate, suck, kill, lose, stupid, bad, as
Neutral	gucci, kane, shawn, madrid, real madrid, murray, sox, milan, open, rousey, anyone, ask
Positive	love, happy, best, good, excite, great, thank, amaze, can wait, enjoy, awesome, fun

Table 5. The most influential words for each class in the Linear SVM.

Negative and positive classes are driven by clear sentiment words. The neutral class relies more on names and topics, which shows the model partly learns topics instead of opinions.

4.4 Predictions on new sentences

Sentence	Prediction
I love this internship!	positive
The class starts at 9am.	neutral
This app keeps crashing, so annoying.	negative
Worst customer service ever, I'm never ordering again.	negative
Not bad at all, I expected worse.	negative

Table 6. Predictions on sentences the model has never seen.

The model handles clear sentences correctly. It struggles with *"Not bad at all, I expected worse"*, which is positive overall but made of negative words. A word-counting model like TF-IDF cannot understand how "not" reverses "bad".

5. Conclusion and future work

I built a complete multi-class sentiment classifier in Python with Scikit-Learn and NLTK. The pipeline covers text cleaning, stop-word removal, lemmatization, TF-IDF features, training three models, and evaluation with F1 scores and a confusion matrix. The best model, a Linear SVM, reached a macro-F1 of 0.592 and accuracy of 0.607 on 12,284 unseen tweets, which is a solid result for a classical model on this difficult real-world dataset.

Future improvements:

- Use pre-trained transformer models such as BERT or RoBERTa, which understand context, negation and sarcasm.
- Add features for emojis and emoticons, which carry strong sentiment in tweets.
- Tune the models on a separate validation set and focus on reducing neutral-class errors.

Appendix: Source code (run in Google Colab)

```
# Task 2: Multi-Class Sentiment Classifier (code as run in Google Colab, cell by cell)

# ---- Step 1: load the data
import pandas as pd, urllib.request
base = "https://raw.githubusercontent.com/cardiffnlp/tweeteval/main/datasets/sentiment/"
def load(split):
    text = urllib.request.urlopen(base + split + "_text.txt").read().decode().splitlines()
    labels = urllib.request.urlopen(base + split + "_labels.txt").read().decode().split()
    return pd.DataFrame({"text": text, "label": [int(x) for x in labels]})
train, test = load("train"), load("test")
print(len(train), len(test))

# ---- Step 2: clean the text (stop-word removal + lemmatization)
import re, nltk
nltk.download("stopwords"); nltk.download("wordnet"); nltk.download("omw-1.4")
from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer

keep = {"no", "not", "nor", "never"}
stop = set(stopwords.words("english")) - keep
lemma = WordNetLemmatizer()

def clean(text):
    text = re.sub(r"\\u2019", "'", text).lower()      # fix escaped apostrophes
    text = re.sub(r"http\S+", " url ", text)         # links
    text = re.sub(r"@w+", " user ", text)            # mentions
    words = re.findall(r"[a-z]+(?:'[a-z]+)?", text)  # split into words
    words = [w for w in words if w not in stop and len(w) > 1]
    words = [lemma.lemmatize(lemma.lemmatize(w, "v"), "n") for w in words]
    return " ".join(words)

train["clean"] = train["text"].apply(clean)
test["clean"] = test["text"].apply(clean)

# ---- Step 3: TF-IDF features
from sklearn.feature_extraction.text import TfidfVectorizer
vec = TfidfVectorizer(ngram_range=(1, 2), min_df=2, max_df=0.9, sublinear_tf=True)
X_train = vec.fit_transform(train["clean"])
X_test = vec.transform(test["clean"])
y_train, y_test = train["label"], test["label"]
print("Train shape:", X_train.shape)
print("Test shape:", X_test.shape)

# ---- Step 4: train and compare three models
from sklearn.naive_bayes import MultinomialNB
from sklearn.linear_model import LogisticRegression
from sklearn.svm import LinearSVC
from sklearn.metrics import f1_score, accuracy_score

models = {
    "Naive Bayes": MultinomialNB(alpha=0.1),
    "Logistic Regression": LogisticRegression(C=2, max_iter=2000, class_weight="balanced"),
    "Linear SVM": LinearSVC(C=0.05, class_weight="balanced"),
}
for name, model in models.items():
    model.fit(X_train, y_train)
    pred = model.predict(X_test)
    print(f"{name:20s} accuracy={accuracy_score(y_test, pred):.3f} macro-F1={f1_score(y_test, pred, average='macro'):.3f}")

# ---- Step 5: evaluate the best model
from sklearn.metrics import classification_report, ConfusionMatrixDisplay
import matplotlib.pyplot as plt

best = models["Linear SVM"]
pred = best.predict(X_test)
labels = ["negative", "neutral", "positive"]
print(classification_report(y_test, pred, target_names=labels, digits=3))

ConfusionMatrixDisplay.from_predictions(y_test, pred, display_labels=labels, cmap="Blues")
plt.title("Confusion matrix: Linear SVM")
plt.savefig("confusion_matrix.png", dpi=200, bbox_inches="tight")
plt.show()

mine = ["I love this internship!", "The class starts at 9am.", "This app keeps crashing, so annoying."]
print(best.predict(vec.transform([clean(s) for s in mine])))
```